

Spatial Continuity Feature

COMPLETE TECHNICAL SPECIFICATION & LIFECYCLE BLUEPRINT

1. Executive Summary & Core Concept

This blueprint details the engineering specifications for a custom, real-time **Spatial Continuity** framework spanning iOS and macOS. The core capability enables a user to directionally "throw" an active context or URL payload from an iPhone to a MacBook via a micro-spatial gesture: **pointing the phone directly at the Mac, placing three fingers on the screen, and swiping upward**. This system prioritizes sub-20ms transfer latency and fluid cross-device exit/entry animations to emulate an uninterrupted physical workflow.

2. Platform Target & Execution Environments

The system is split into two co-dependent targets constructed exclusively inside **Xcode** using **Swift**, **SwiftUI**, and native Apple subsystem frameworks:

- **macOS Client (Background Agent App):** Configured as a background utility with no persistent Dock visibility. This is accomplished by specifying `LSUIElement = YES` (Application is agent) in the project's `Info.plist`. It exposes user control exclusively through a Menu Bar icon interface.
- **iOS Client (Application & System Extension):** Implemented as a primary container app combined with a native **iOS Share Extension** or **Action Extension**. This extension bypasses background sandbox touch limits, scraping active context payloads (e.g., Safari or YouTube URLs) and capturing multi-touch gestures directly inside an active share interface overlay.

3. macOS Lifecycle, Menu Bar UI & Battery Optimization

To eliminate unnecessary CPU wake cycles and radio overhead on the Mac, the lifecycle must be explicitly managed:

Menu Bar Integration (NSStatusItem)

The macOS app instantiates a native `NSStatusItem` in the system menu bar. Clicking this icon reveals an `NSMenu` dropdown providing status diagnostics, preferences, and an explicit terminate control linked directly to `NSApp.terminate(nil)` to completely stop background execution.

Power State & Radio Optimization

To preserve battery life, the high-power **Nearby Interaction (Ultra Wideband)** session must not run continuously. The macOS background daemon stays in a low-power Bluetooth Low Energy (BLE) peripheral mode. The iPhone serves as the coordinator: when the iPhone screen unlocks and detects the Mac's proximity via BLE, it transmits an active wake packet over local ad-hoc channels, prompting both apps to fire up their respective Ultra Wideband hardware *only* during active utilization windows.

4. Technical Core Frameworks

- **Nearby Interaction (NI):** Interfaces with the U1/U2 Ultra Wideband silicon to extract a 3D directional vector (x, y, z) between devices. A relative angle tolerance check determines alignment:

$$\cos(\theta) = (\text{Vector_A} \cdot \text{Vector_B}) / (||\text{Vector_A}|| * ||\text{Vector_B}||)$$

If $\theta \leq \pm 15^\circ$, spatial validation is approved.

- **Multipeer Connectivity:** Spins up a low-latency infrastructure Wi-Fi / Peer-to-Peer Wi-Fi and BLE socket mesh. Data frames are serialized into small binary chunks and dispatched instantly using `MCSession.send(_, toPeers:, mode: .reliable)`.
- **App Automation (macOS):** Executes incoming actions system-wide using `NSWorkspace.shared.open(url)` to manipulate default browsers or media applications seamlessly.

5. Implementation Checkpoints for Code Generation

1. **Checkpoint 1: Local Network Mesh (P2P):** Build the ad-hoc Multipeer Connectivity loop in Xcode for both platforms. Verify bidirectional text packet synchronization completes under a 30ms baseline.
2. **Checkpoint 2: Background Agent Configuration:** Configure the macOS project `Info.plist` with `LSUIElement = YES`. Program the `NSStatusItem` menu loop to instantiate the Wi-Fi-style status bar drop menu containing a functional "Quit" termination action.
3. **Checkpoint 3: Spatial Awareness Tracking:** Implement the `NISessionDelegate` across both entities. Log coordinates and establish structural trigonometric filters validating real-time targeting constraints.
4. **Checkpoint 4: Gesture Binding & Payload Dispatch:** Bind a 3-touch `UIPinchGestureRecognizer` and a `UISwipeGestureRecognizer` inside the iOS extension layout to scrape contextual URLs, execute the SwiftUI exit transition, and dispatch payloads to the target.
5. **Checkpoint 5: System Open & Entry Animation:** Wire up the Mac receiver to execute incoming URLs programmatically while presenting an overlay card sliding smoothly from the bottom display border.